# Design layer

the whole layer, as one document

# Contents


**1 Gleam Dream ecosystem ..... 4**

1.1 Foundation ..... 4
1.2 System at a glance ..... 5
1.2.1 Package responsibilities and dependencies ..... 5
1.3 Contexts ..... 6

**2 Composition applications ..... 9**

2.1 Foundation ..... 9
2.2 Pending updates ..... 9
2.3 Composition applications ..... 9
2.3.1 Session replay ..... 10
2.3.2 Checkout ..... 11
2.3.3 Financial recovery ..... 11
2.3.4 Webhook delivery ..... 14
2.3.5 Document extraction ..... 15
2.3.6 Support desk ..... 15
2.3.7 Tool hub ..... 16
2.3.8 Protected MCP service ..... 16
2.3.9 Email portal ..... 17
2.3.10 SSO portal ..... 17
2.3.11 Durable research and publication ..... 18

**3 Composition contracts ..... 21**

3.1 Foundation ..... 21
3.2 Pending updates ..... 21
3.3 Caller contracts ..... 22
3.3.1 Values and definitions ..... 22
3.3.2 Extensions retain shared guarantees ..... 22
3.4 Execution and evidence boundaries ..... 23
3.4.1 Delivery is not effect settlement ..... 23
3.4.2 Identity scopes remain separate ..... 24
3.4.3 Budgets and cleanup have different intervals ..... 24
3.4.4 Pool and process lifetimes are explicit ..... 24
3.5 Protocol, schema and authorization contracts ..... 25
3.6 Observation and diagnostic contracts ..... 26

**4 Evidence and validation ..... 29**

4.1 Foundation ..... 29
4.2 Pending updates ..... 29
4.3 Validation commands ..... 29
4.4 Source qualification ..... 30
4.5 Experiments and acceptance evidence ..... 31
4.5.1 Interface laboratory ..... 32
4.5.2 Admission deadline and capacity ..... 33
4.5.3 Authorization and approval ..... 33
4.5.4 Irreversible effects and replay ..... 34
4.6 Design maintenance and rendering ..... 34

**Glossary ..... 35**

# Gleam Dream ecosystem

# 1 Gleam Dream ecosystem

## 1.1 Foundation

Oversight owns the composition contracts and executable evidence that connect independent Gleam libraries. Each context owns the rest of its foundation.

GOAL

### Compose independently useful libraries

Applications build complex behavior from small public surfaces backed by cohesive capabilities. Each library remains useful without an ecosystem-wide runtime.

GOAL

### Retain the intended capability scope

A bounded release surface does not erase the broader Capability scope. Unbuilt requirements remain visible in the package that owns them.

NO-GOAL

### A shared application framework

Oversight supplies no mandatory runtime, service container, universal error type, common retry engine, or global policy registry.

INVARIANT

### Every contract has one semantic owner

Each Package authority defines its own guarantees. Consumers translate meanings explicitly; similar record fields alone do not establish shared meaning.

convention

PRINCIPLE

### Simplicity includes application responsibility

Prefer the design that removes unnecessary concepts and repeated application decisions while retaining meaningful caller control. A shorter facade that shifts hidden work to every application is not simpler.

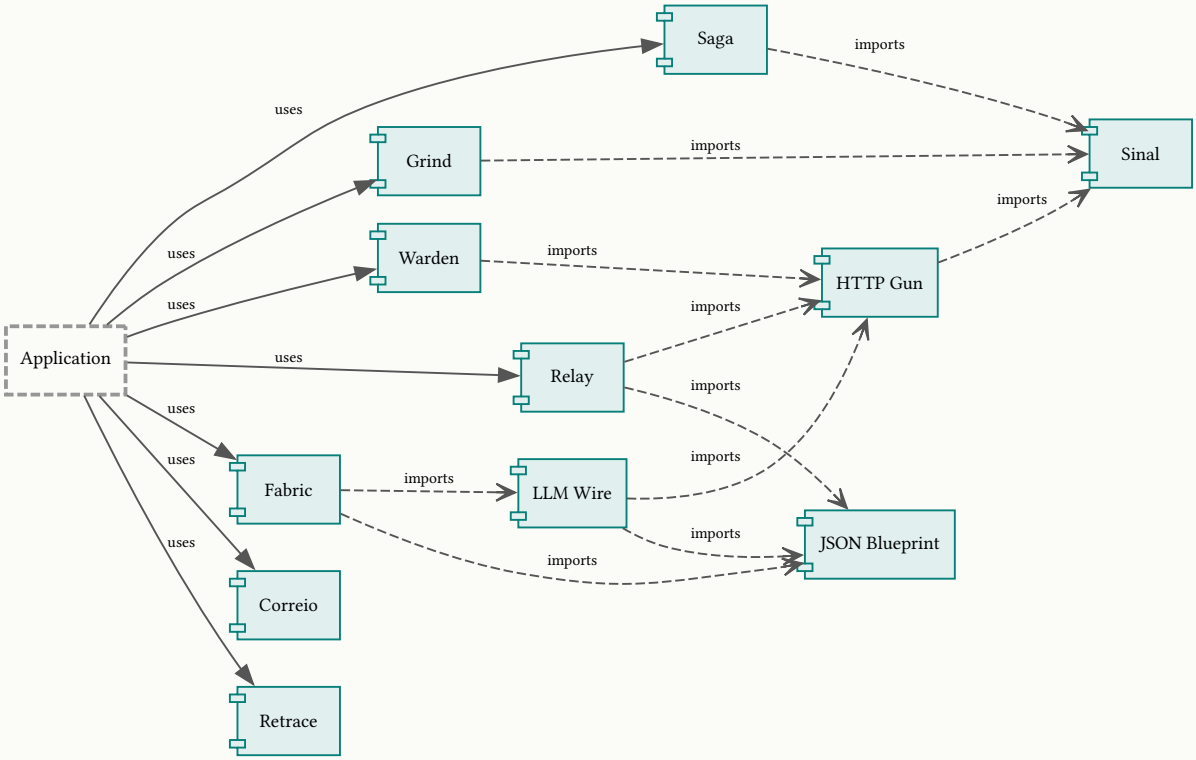
composition

The documentation ownership decision is recorded in ADR-0001. The package and API boundaries are justified by ADR-0002 and ADR-0003.

## 1.2 System at a glance

The ecosystem contains independent libraries, four PostgreSQL adapters, a Postmark adapter, a development mailbox adapter and Retrace core, frontend and server packages. Oversight supplies design coordination and executable composition evidence, with no production runtime dependency.

VIEWPOINT RUNTIME · ALTITUDE L1 · BOUNDARY Applications select independent library authorities




LEGEND




*The map shows the principal runtime dependencies. The package table lists every direct sibling dependency. The Retrace node groups its independently selected core, frontend and server packages. PostgreSQL adapters borrow an application-owned connection pool.*

### 1.2.1 Package responsibilities and dependencies

The table lists every direct sibling dependency and its semantic exclusion.

| Package | Owns | Direct sibling dependencies | Excludes |
|---|---|---|---|
| Sinal | Typed observations and correlation | None | Export policy and application control flow |
| JSON Blueprint | Codecs, schema projection, runtime contracts | None | Business authorization and execution |
| HTTP Gun | HTTP transport, body ownership, redaction and cassettes | Sinal | Provider semantics and automatic application retry |
| Saga | Typed workflows, compensation and effect settlement | Sinal | Job delivery and provider-specific reconciliation knowledge |
| Grind | Durable job admission, scheduling and delivery | Sinal | Application workflow meaning and exactly-once external effects |

| Package | Owns | Direct sibling dependencies | Excludes |
|---|---|---|---|
| LLM Wire | Provider protocols and classification | HTTP Gun, JSON Blueprint, Sinal | Conversation ownership and autonomous tool execution |
| Relay | MCP client/server contracts and transport | HTTP Gun, JSON Blueprint, Sinal | OIDC custody and application authorization |
| Warden | OIDC/OAuth client behavior and token custody/validation | HTTP Gun, Sinal | MCP-specific policy and application sessions |
| Correio | Transactional message construction, delivery evidence and passwordless verification | None | Account policy, application sessions and OIDC token claims |
| Fabric | Agents, durable graphs, approvals and managed children | LLM Wire, HTTP Gun, JSON Blueprint, Sinal | Job delivery and Saga effect semantics |
| Retrace core | Typed recording, checkpoints, wire validation and pure reconstruction | None | Capture policy, UI and storage ownership |
| retrace_frontend | Browser/Tauri delivery, Lustre capture and optional replay viewer | Retrace core | Application state, safe projections and historical build publication |
| retrace_server | Session admission, indexing and object archival | Retrace core | Pool/worker lifetime, authentication and retention policy |
| saga_postgres | Saga persistence, leases and fencing | Saga, Sinal | Pool ownership and driver scheduling |
| fabric_postgres | Fabric persistence and family retention | Fabric | Pool ownership and model/tool policy |
| warden_postgres | Warden sealed-record persistence and conditional writes | Warden, HTTP Gun, Sinal | Pool ownership, sealing keys and application authorization |
| correio_postgres | Atomic challenge persistence and retention | Correio | Pool ownership and account policy |
| correio_postmark | Postmark submission and delivery evidence | Correio, HTTP Gun | HTTP client ownership and automatic resend |
| correio_mailbox | Read-only development inspection of captured mail | Correio | HTTP server ownership and production access policy |

- Each package repository owns its detailed layer. Oversight references those contracts without duplicating the package's state machines or glossary.
- The application selects the participating libraries; no package imports Oversight at runtime. The package set is recorded in ADR-0014, ADR-0016 and ADR-0017 removed bridge rationale is recorded in ADR-0003.
- A Consumer recipe may join packages absent from each other's dependency graph. The recipe must understand their public semantics, not their private representation.

## 1.3 Contexts

Three contexts divide the layer by vocabulary. Each context owns its terms, and a context cites another context's term without redefining it.

| Context | Owns | Vocabulary |
|---|---|---|
| composition | Caller contracts, execution and evidence boundaries, protocol and authorization composition, observation joins | Package authority, composition boundary, effect evidence, admission identity |
| applications | The composition applications and each application's task, ownership and recovery behavior | Composition application, business identity, reconciliation decision, applied marker |

| Context | Owns | Vocabulary |
| --- | --- | --- |
| evidence | Retained experiments, validation profiles, source qualification and design maintenance | Public consumer, behavioral oracle, locked source, validation profile |

- applications conforms to the contracts of composition each application demonstrates one or more composition boundaries.
- evidence judges both other contexts and defines what a passing check does and does not establish.
- The relationships between contexts are declared in the context map.

CONTEXT

# Composition applications

# 2 Composition applications

## 2.1 Foundation

Each application is a separately compiled consumer that completes one task across several package authorities, using its own domain types, policies, startup and shutdown.

GOAL

### Complete real tasks through public contracts

An application performs an ordinary task, an advanced configuration, native-value handling and a failure recovery using only public package modules.

GOAL

### Expose recovery paths without credentials

A Scripted peer and a disposable database reproduce lost replies, refusals, process loss and cancellation locally, so recovery behavior is executable without provider accounts.

NO-GOAL

### A production deployment template

An application demonstrates a composition. It is not a complete ledger, account system, session transport, accounting specification or compliance assessment.

INVARIANT

### Application records own application facts

Payment, order, session and release facts live in application-owned records. Library records establish delivery, workflow or conversation state and never replace them.

convention

PRINCIPLE

### A recipe translates and never extends a contract

A Consumer recipe maps one package's public result into another's input. It adds no state machine, retry engine or bridge dependency of its own.

composition

The application set and its package boundaries are recorded in ADR-0003, ADR-0010, ADR-0013, ADR-0016, ADR-0017 and ADR-0018.

## 2.2 Pending updates

ruling **Provider contract lacks proof of non-execution** 2026-10-07

Releasing an unconfirmed payment reservation needs a provider contract that excludes late execution. The scripted provider supplies lookup evidence only for completed payments, so an unavailable lookup or an absence never releases a reservation. See ADR-0011.

verify **Application invariants are unchecked against the apps** 2026-10-17

The invariants and behavior rules for the compiled applications were folded from retired application READMEs and feedback notes. Each needs a comparison against the application source and its tests before the section counts as verified.

## 2.3 Composition applications

Each application is a separately compiled consumer with its own domain types, policies, startup, and shutdown. Scripted peers expose deterministic failure paths without production credentials.

### Composition application

aggregate · stateful · composition · owned by Oversight

One executable consumer demonstrating a complete application task across selected package authorities.

ATTRIBUTES

authored **Scenario contract** · Named task and observable outcomes
The ordinary task, advanced case, native values and failure cases the consumer establishes.

authored **Package configuration** · Public typed configuration values
The participating packages and policies chosen by the application.

observed **Evidence** · Observed assertions and outcomes
Results of actually executed scenarios, with toolchain and environment limits attached to the claim.

RELATIONSHIPS

`1 : n` A composition application exercises multiple package authorities, each of which may participate in several applications.

## 2.3.1 Session replay

### Application-owned cart reconstruction

**responsibility**

Demonstrate portable recording and pure reconstruction without a browser, database or native shell.

**interface**

The separate public consumer under `apps/session_replay` supplies Cart, Message and CartError types, a pure reducer, an explicit session contract and elapsed times.

**interactions**

The application accepts positive additions/removals without a negative cart quantity, then records each accepted message. Retrace numbers entries and reconstructs state. The application may resume at a trusted checkpoint and append further accepted messages.

**invariants**

Only accepted state transitions enter the stream. Replay runs the same pure reducer and performs no external effects. A failed step leaves the previous player available. An in-memory checkpoint is not proof of durable persistence.

**failure**

Missing entries and incompatible contracts are refused explicitly. Invalid quantities and removal beyond the cart retain native domain errors; the application can replay a corrected contiguous sequence or start from a trusted later checkpoint.

BEHAVIOR

### Accepted cart messages reconstruct the same state

**Given** A safe initial cart and its accepted messages, or a trusted checkpoint and the accepted messages following it.

**When** The application reconstructs the contiguous stream with its matching contract and pure reducer.

**Then** The reconstructed cart equals the application's recorded cart at the same cursor.

level: boundary · area: Session replay

BEHAVIOR

### Replay refusal preserves a recovery point

**Given** A replay encounters a gap, incompatible contract or a message refused by the cart reducer.

**When** The consumer attempts reconstruction.

**Then** It receives a typed failure without advancing the previous player, so it can retry from that recovery point with valid input.

level: boundary · area: Session replay

- This consumer establishes core behavior only. Retrace owns the maintained basic browser and advanced Tauri examples and their separate UI, delivery and storage evidence.

ADR-0016

### 2.3.2 Checkout

#### Checkout authority and recovery

**responsibility**

Accept orders, reserve stock, charge payment, book shipment and enqueue receipts.

**interface**

Order placement and lookup; local execution and a durable-job mode with native order/receipt/error codecs.

**interactions**

Saga orders effects and undo; HTTP Gun calls scripted payment/shipping peers; Grind delivers durable runs and receipts; saga_postgres and application rows use the shared pool.

**invariants**

Order insertion and durable-job admission share one application transaction. Confirmation and receipt admission share a transaction. Stable destination keys and explicit unknown_when policy preserve payment uncertainty.

**failure**

Known failures compensate in reverse completion order. Lost driver ownership permits checkpoint resumption; requested job cancellation is translated into durable Saga cancellation and rollback.

- The application owns payment/shipment reconciliation and release policy. A job retry alone cannot decide whether stock or money should be restored.
- Source owners: `apps/checkout/src/checkout/workflow.gleam`, `orders.gleam`, `jobs.gleam`, `gateway.gleam`, and `db.gleam`.

### 2.3.3 Financial recovery

#### Payment evidence across independent durable records

**responsibility**

Retain application financial intent, reservations, payment evidence and local postings while coordinating recoverable execution and job delivery.

**interface**

A separate public consumer under `apps/financial_recovery`, with native operation identifiers, payment requests, outcomes and errors. Its test entry runs against disposable PostgreSQL and a scripted loopback payment provider.

**interactions**

The payment record owns financial facts. Saga owns the step checkpoint and interrupted-effect resolver. Grind owns delivery and attributed replay authorization. HTTP Gun supplies exchange evidence without inferring provider execution.

**invariants**

A stable business operation identity survives job and receipt pruning. Reservation remains held while the payment result is unknown. Accepted evidence and local posting are idempotent by business identity. Re-entering a completed operation does not send another payment or post again.

**failure**

A lost response or unavailable lookup leaves an unresolved payment. An interrupted Saga action uses its resolver before another effect is permitted. A completed unresolved Saga outcome is retained as such; coordinator replay does not turn it into an interrupted action.

| Record | Owner | Stable identity | Authority |
|---|---|---|---|
| Payment operation | Application | Business operation identifier | Intent, reservation, evidence and posting |
| Workflow execution | Saga | Run and effect identifiers | Admitted actions, checkpoints and interrupted-step recovery |
| Delivery | Grind | Submission, job and attempt identifiers | Scheduling, acknowledged outcome and attributed resolution |

- The consumer uses one application-configured Grind pool; application queries and saga_postgres borrow it. Startup applies all required schemas before admitting jobs. Shutdown stops drivers and borrowers before the pool.
- READ COMMITTED admission locks known account rows in a deterministic order and atomically records financial intent, reservation and job insertion. The locking protocol protects the invariant that balance is at least reserved and reserved is not negative for those known rows; it does not cover arbitrary predicates, new account sets or writers that bypass the protocol, and it is not a universal replacement for SERIALIZABLE isolation. A second admission over a locked account waits, then is refused with an insufficient-funds failure that leaves no operation or job.
- A Business identity identifies one payment operation; Saga and queue identities derive from it. Reuse with changed intent is refused as an identity conflict, and admission of a completed operation returns the retained result before Saga or Grind is consulted. The provider lookup encodes the identity as one path segment while records and the idempotency header keep it unchanged. An unconfirmed outer transaction preserves uncertainty: the caller reads the application record or retries admission under the same identity and never generates a replacement.
- The coordinator holds every non-completed Saga outcome for inspection, so a held job may carry a known refused effect as well as an unknown effect. The retained Saga outcome is inspected before an operator decision is chosen.
- A caller retaining SERIALIZABLE isolation receives Grind's typed refusal for `submit_in`. An application outbox can atomically record dispatch intent with its business transaction and submit separately under a stable operation key. It requires explicit dispatch, retry and retention ownership rather than an implied atomic cross-transaction enqueue.
- An audited AuthorizeReplay command permits another coordinator delivery only where cancellation policy allows it. The coordinator resumes the same durable execution and resolves interrupted effects from authoritative evidence; the authorization does not itself permit another payment request.
- A provider lookup must distinguish confirmed completion, authoritative non-execution and insufficient evidence. An unavailable lookup or a non-authoritative absence does not release a reservation or authorize another external effect.
- A local posting transaction and the later Saga checkpoint remain separate commits even when they use the same database. The posting record supplies recovery evidence; the consumer does not add a second Saga storage protocol.
- For a finished Unresolved execution, application reconciliation binds the retained coordinator job and validates its native input against financial intent. Positive provider evidence permits one transaction to record the evidence, posting and attributed reconciliation decision. Unavailable, absent, malformed or conflicting evidence cannot release the reservation. The provider contract supplies no authoritative non-execution proof.
- The coordinator's business obligation is a confirmed payment and local posting. After the application settlement commits, attributed Grind confirmation records that obligation's fulfillment without changing the original Saga report. Cancellation does not prevent recording an executed payment; reconciliation never initiates another payment or a refund.
- A Reconciliation decision binds one job, operation, actor, reason and accepted reference under one decision identity. Exact retries reuse the committed decision without another lookup. Pending application records permit bounded discovery with a decision-identity cursor and resumption by decision identity; the caller owns authorization, scheduling and retention. A blocked record does not hide later records, and no recovery service starts implicitly.
- After financial settlement, queue confirmation and its local acknowledgment share one short caller-owned READ COMMITTED transaction through Grind's borrowed resolution API. A failed local acknowledgment rolls back queue resolution; a committed acknowledgment survives queue pruning under application retention. A lost outer commit reply requires application readback or resumption by the same decision identity. Investigation remains outside this transaction, whose locks last until commit or rollback. Missing queue history and separate operator resolutions remain explicit errors; neither creates another delivery or fabricates acknowledgment. See ADR-0012.
- The scripted provider owns independent payment evidence and submission counts. Process barriers expose effect/checkpoint gaps. Loopback lost-response evidence establishes HTTP delivery behavior, not TLS or live-provider qualification.

BEHAVIOR

### Financial admission commits with delivery

**Given** An application admits a payment operation inside a supported transaction.

**When** The transaction commits or rolls back after job insertion.

**Then** Financial intent, reservation and job admission become visible together or remain absent together.

**Then** An incompatible reuse of the business identity is refused without another reservation.

**Then** Competing admissions over the same locked accounts cannot reserve more than their available balance; refusal leaves no partial operation or delivery.

level: boundary area: Financial recovery

BEHAVIOR

### Payment execution checks application intent

**Given** A caller invokes the coordinator through Saga or Grind.

**When** The transfer effect checks the application's payment record.

**Then** Missing or incompatible intent prevents a provider submission.

**Then** Retained completion evidence supplies the known result without another provider submission.

level: boundary area: Financial recovery

BEHAVIOR

### Interrupted payment recovery preserves uncertainty

**Given** A payment action is interrupted after the provider may have executed it and before Saga checkpoints its result.

**When** The replacement coordinator cannot obtain authoritative provider evidence.

**Then** Saga recovery remains suspended and Grind exposes an uncertain job outcome.

**Then** The application retains the reservation and does not submit another payment.

level: boundary area: Financial recovery

BEHAVIOR

### Authorized delivery resolves before another effect

**Given** An interrupted payment has authoritative completion evidence and no pending cancellation.

**When** An attributed resolution authorizes delivery of its coordinator again.

**Then** The same Saga execution resolves the interrupted action and the application posts the known payment once.

**Then** Repeated evidence or completed-operation re-entry does not send another payment or create another posting.

level: boundary area: Financial recovery

BEHAVIOR

### Local posting survives a missing checkpoint

**Given** The posting transaction commits before Saga checkpoints step completion.

**When** The executing process dies and another coordinator recovers the same operation.

**Then** The resolver discovers the committed posting and supplies its result.

**Then** The posting callback is not repeated.

level: boundary area: Financial recovery

BEHAVIOR

### Cancellation preserves financial uncertainty

**Given** A payment effect is unresolved.

**When** Cancellation and its uncertain worker acknowledgement arrive in either order.

**Then** The job's uncertainty and evidence remain queryable and its cancellation intent prevents replay authorization.

**Then** The application retains its reservation until explicit financial reconciliation settles the operation.

level: boundary area: Financial recovery

BEHAVIOR

### Business identity outlives queue retention

**Given** A payment operation and posting are complete and the old terminal job and its receipts have been pruned.

**When** The application receives the same business operation again.

**Then** The application returns its retained result without another payment or posting.

level: boundary area: Financial recovery

BEHAVIOR

### Finished uncertainty admits application settlement

**Given** A coordinator job is uncertain and its Saga has a saved Unresolved report.

**When** An authorized application reconciliation obtains authoritative payment completion evidence.

**Then** Evidence, one local posting and an attributed decision commit together before delivery is reported as confirmed.

**Then** The original Saga report remains unchanged; repeated or concurrent identical decisions cannot post or submit the payment again.

**Then** A missing, malformed or conflicting confirmation leaves the reservation held. A posting failure rolls back the application settlement and cannot produce a confirmed delivery outcome.

level: boundary area: Financial recovery

BEHAVIOR

### Reconciliation resumes between independent commits

**Given** An application settlement committed before the reconciler process stopped.

**When** The caller discovers the pending decision and resumes it.

**Then** The retained decision supplies the exact attributed delivery resolution without another provider lookup or financial effect.

**Then** Delivery resolution and the local acknowledgment become durable together; rollback preserves the pending decision and uncertain delivery.

**Then** An already committed acknowledgment remains readable after delivery pruning.

**Then** Missing delivery history retains an explicit pending acknowledgment and the settled financial facts.

level: boundary area: Financial recovery

BEHAVIOR

### The application retains its isolation choice

**Given** The application supplies a SERIALIZABLE transaction to Grind admission.

**When** The consumer attempts `submit_in`.

**Then** The typed unsupported-isolation result is preserved and the application transaction is not silently weakened.

level: boundary area: Financial recovery

ADR-0010 ADR-0011

## 2.3.4 Webhook delivery

### Webhook delivery policy

**responsibility**

Fan application events out to active tenant subscriptions as signed durable deliveries.

**interface**

Subscription/event operations and a contextual Grind worker with a stable delivery identity.

**interactions**

Application rows and enqueue share a transaction. HTTP Gun applies per-tenant destination views; Grind executes explicit retry/snooze/discard policy.

**invariants**

The same delivery id signs and identifies every retry. Registration and delivery use destination policy; delivery re-reads subscription configuration so disablement and key rotation are effective.

**failure**

Successful status can settle delivery despite an oversized unused body. Rate limits snooze within the cap; gone/unsafe destinations disable subscriptions; retryable transport/server failures consume bounded attempts.

- Destination policy is per tenant tier on one HTTP client. A public tenant reaches public addresses only and every DNS answer is checked; the internal tier reaches allowlisted loopback host entries. Registration applies the same policy value, so a refused address is rejected before any delivery.
- A response body beyond the read limit keeps its success status, so delivery settles once. A rate-limit response snoozes for the advised interval up to a configured cap and then consumes an attempt.
- Worker tests replay recorded refusals and body-limit failures from cassettes without a receiver, and a request that skips the tenant's client view is refused before sending.

Source owners: `apps/webhooks/src/webhooks/attempt.gleam`, `delivery.gleam`, `subscription.gleam`, `egress.gleam`, and `web.gleam`.

### 2.3.5 Document extraction

**Typed document extraction**

**responsibility**

Fetch bounded documents, obtain a typed invoice from a provider and persist the result.

**interface**

Batch job submission and native Invoice/LineItem/Currency values with semantic validation.

**interactions**

Separate HTTP clients carry document and provider policies. LLM Wire uses one Blueprint codec for structured output; small adapters reuse the codec for Grind and database values.

**invariants**

Document content, declared length and collected bytes are bounded before model input. Unsupported content and invalid invoices do not become successful stored outputs.

**failure**

The application translates fetch, provider, validation and storage failures into retry, snooze or discard. Provider advice does not directly own job policy.

Source owners: `apps/extractor/src/extractor/invoice.gleam`, `document.gleam`, `llm.gleam`, `outcome.gleam`, and `jobs.gleam`.

### 2.3.6 Support desk

**Approved compensating refunds**

**responsibility**

Run a support agent whose refunds require an authorized approval and a compensating workflow.

**interface**

Ticket/run operations, typed tools, typed Resolution, and approve/reject commands carrying checked proof.

**interactions**

Fabric owns agent state and approval; Warden checks approver identity; Saga performs refund/ledger/notification; the consumer recipe retains full reports before mapping them.

**invariants**

The proof comes from the configured verifier and required scope. Ticket correlation reaches model, tool, Saga and HTTP calls. Reopening a ticket converges on its run.

**failure**

Unauthenticated, unauthorized and unavailable-verifier outcomes are separate. Unknown payment effects require reconciliation; failed notification triggers the workflow's compensation policy.

Source owners: `apps/support_desk/src/support_desk/desk.gleam`, `approvers.gleam`, `refund.gleam`, `domain.gleam`, and the local Saga recipe.

### 2.3.7 Tool hub

#### MCP in both directions

**responsibility**

Let an agent call discovered MCP tools and expose agent execution as an MCP tool.

**interface**

Relay declarations/calls, admitted dynamic contracts, Fabric invocation, and application recipes.

**interactions**

The application starts the inventory peer it discovers. Relay carries idempotency/correlation metadata; Fabric owns durable keyed invocation.

**invariants**

Dynamic schema rejection is explicit. The local peer's trusted read-only declaration is an application precondition, not a trust rule for arbitrary external peers.

**failure**

Keyed disconnect can leave durable work inspectable; unkeyed cancellation stops its owned invocation. Tool failure and uncertain delivery remain separate from a completed protocol exchange.

Source owners: `apps/tool_hub/src/tool_hub.gleam`, `relay_discovery.gleam`, `relay_run.gleam` and `relay_serve.gleam`, and `relay_tools.gleam`.

### 2.3.8 Protected MCP service

#### Authenticated tools and durable reports

**responsibility**

Serve MCP tools for authenticated principals and delegate long report work to jobs.

**interface**

Browser login, bearer-checked MCP requests and report submission/readback through public app routes.

**interactions**

Warden supplies token validation/introspection; Relay owns protected-resource admission; the app derives principal and tenant policy; Grind owns report delivery.

**invariants**

Token issuer/audience/scope checks precede tool business effects. Application ownership of report data remains distinct from a valid token.

**failure**

Missing permission is a refusal, not a provider outage; unavailable verification does not admit the request. Job failures remain typed at readback.

- The login asks for the resource of the MCP endpoint, and the provider issues access tokens whose audience is that endpoint. Local validation checks signature, type, issuer, audience and expiry; introspection is an alternative mode that also reaches the provider. A revoked token that local validation accepts is an explicit limit of local mode.
- A job owned by another user answers exactly like a missing job.

Source owners: `apps/secure_mcp/src/secure_mcp/auth.gleam`, `tools.gleam`, `reports.gleam`, and `web.gleam`.

### 2.3.9 Email portal

#### Delivery, inspection and application sessions

**responsibility**

Retain a complete email sign-in recipe under apps/email_portal using public Correio contracts.

**interface**

Application-native account and session types, explicit challenge request and confirmation operations, capture checkpoints and the standard Gleam HTTP mailbox handler.

**interactions**

Correio's memory store owns atomic challenge state. Capture retains local mail. The application owns both runtimes, its account policy, pending verification evidence and idempotent Session publication.

**invariants**

The mailbox route is explicitly configured for development. Pending values and confirmed evidence are not application sessions. A disabled or changed account cannot receive a session. Repeated publication of one verification identity returns the same session.

**failure**

Delivery refusal, unknown delivery, invalid confirmation and failed publication remain distinct application outcomes. Publication recovery uses retained evidence and rechecks current account policy.

- The ordinary flow takes a checkpoint, requests delivery, inspects the resulting message through the mailbox handler, explicitly confirms its secret and reads the application session. Advanced use configures a different mount path and challenge policy with email codes.
- Tests exercise separate consumer compilation, actual capture and memory runtimes, and in-process HTTP handler requests. They do not claim browser navigation, network transport, durable session storage or production authentication policy. Correio owns its separate browser, transport, database and upstream-oracle evidence.
- Detailed library design remains in the Correio repository. Oversight owns this composition and its scenario assertions. ADR-0017
- Session state in this application is local and sequential. A production publisher needs durable atomic idempotency and an application-owned session credential, and pending evidence belongs to this service context and is never accepted from untrusted clients or other application instances.

BEHAVIOR

#### Mailbox inspection grants no session

**Given** A sign-in request has delivered a challenge into the development capture.

**When** The application inspects the mailbox and message.

**Then** The challenge remains available for intentional confirmation.

**Then** No application session exists before successful confirmation.

level: boundary area: Email portal

BEHAVIOR

#### Session publication recovers confirmed verification

**Given** Challenge consumption has succeeded but application session publication failed.

**When** The application retries publication with the retained verification evidence.

**Then** Current account policy is checked again.

**Then** Publication creates or recovers one session without a second challenge consumption.

level: boundary area: Email portal

### 2.3.10 SSO portal

#### Durable login custody in a web application

**responsibility**

Maintain browser sessions and call a protected billing resource with recoverable token custody.

**interface**

Login/callback/logout routes, application session cookie, and native principal/account views.

**interactions**

Warden owns transactions and token custody; the application implements Warden's conditional sealed-record writes on PostgreSQL and separately manages its browser session. HTTP Gun performs resource calls.

**invariants**

The application does not implement a second refresh state machine. Custody records and browser session references have separate owners and lifetimes. Guarded consumption checks version and database wall time under row ownership after lock waiting; validity and storage retention remain separate.

**failure**

The application may refresh once after a resource refusal and recover uncertain publication once, then maps Warden's typed action to user-visible decisions. Storage uncertainty is not treated as definite non-action. Listener startup refusal returns a recoverable error without terminating its caller. Listener shutdown and caller exit retire the owned Mist subtree.

- Warden's stored records are sealed with an application-held key and hold nothing readable. The PostgreSQL adapter implements Warden's explicit write conditions: a guarded write locks the row, checks its version and samples database time before updating, and a failed commit acknowledgement remains an unknown outcome.
- Web-tier restart waits for the owning actor to retire the previous listener subtree, allowing a bounded interval for child cleanup and failing explicitly on timeout.

BEHAVIOR

### Listener startup refusal preserves its caller

**Given** Another listener already owns the requested address.

**When** The application attempts to start its listener.

**Then** Startup returns an error without terminating the caller.

**Then** The caller can start and stop a listener at an available address.

level: boundary area: SSO portal

ADR-0018 Source owners: `apps/sso_portal/src/sso_portal/warden_records.gleam`, `session_store.gleam`, `web.gleam`, `billing_api.gleam`, and `server.gleam`.

### 2.3.11 Durable research and publication

## Independent delivery, conversation and publication

**responsibility**

Resume research across restarts and publish its result through a compensating workflow.

**interface**

A Grind worker starts or resumes a Fabric agent with search/read tools, then drives a durable Saga publication workflow.

**interactions**

Fabric persists conversation and tool results; Grind schedules delivery; Saga owns publication effects; their PostgreSQL adapters share application resources.

**invariants**

A fresh driver resumes durable state rather than replaying an arbitrary closure. Read-only tool replay is explicit. Publication keys and provider readback settle external effects.

**failure**

Transient driver/store conflicts snooze; domain failures follow application retry policy; unresolved publication becomes Uncertain. Unexpected approvals fail explicitly. Cancellation reaches the agent/model and publication workflow without proving that every external effect was undone.

- A rejected publication is definite only when its draft-deletion undos succeed; a failed cleanup remains uncertain. Draft deletion is idempotent and accepts an already-deleted draft.
- On interruption, creation looks up its idempotency key. Upload and publication read the draft's committed state: an uploaded or live draft proves upload, and a live draft with its address proves publication. Missing, malformed or unavailable readback cannot exclude a late commit and never authorizes another mutation. The application requires this readback contract from its publisher; idempotence alone is not evidence that an earlier operation was never sent.
- Publication carries a step version and a versioned error codec. A checkpoint written under an earlier step version fails closed as incompatible, including a finished failure, because the earlier error lost effect evidence that its text cannot restore. Such records are reconciled from provider evidence before an operational migration; they are neither deleted nor relabeled as retryable.

ADR-0024 Source owners: `apps/research_agent/src/research_agent/jobs.gleam`, `publish.gleam`, `researcher.gleam`, `node.gleam`, and `wire.gleam`.

### History-aware turn delivery

**responsibility**

Demonstrate application-owned turn data across durable admission, Fabric execution and outcome incorporation.

**interface**

A separate public consumer in the research application uses Grind transactional admission, Fabric history-aware keyed invocation, the leased PostgreSQL store and its registered recovery sweeper.

**interactions**

The application transaction records a turn and its delivery together. Delivery supplies validated history and the current prompt to Fabric. The sweeper resumes admitted execution after process loss; an idempotent application transaction incorporates the native answer and generated messages.

**invariants**

The stable principal and turn key identify execution; correlation and live context do not replace that identity. Retain the outcome until incorporation commits. A durable Applied marker prevents a later delivery from reinvoking an incorporated turn after Fabric pruning.

**failure**

Working, suspension, uncertain execution, ended failure and refusal retain distinct application decisions. Lost incorporation acknowledgement requires readback or the same idempotent transaction. A shared connection proves no atomicity between the application transaction and Fabric's initial record.

- This consumer uses existing Grind delivery and Fabric recovery. It adds no application lease renewal, abandoned-run scanner or competing generic queue.
- A per-turn application graph may instead own incorporation as a declared operation with explicit idempotency and interrupted-effect recovery. It still needs the transactional admission bridge and does not imply a new PostgreSQL admission API.
- Deterministic model callbacks and disposable PostgreSQL establish public composition and tested process-loss behavior. They establish no live-provider or power-loss guarantee.

BEHAVIOR

### Applied turns survive execution record pruning

**Given** The application has incorporated an agent outcome and retained its applied marker.

**When** Delivery repeats after Fabric's settled run record is pruned.

**Then** The application returns the incorporated result without starting another agent run.

level: boundary area: Durable research

ADR-0013

CONTEXT

# Composition contracts

# 3 Composition contracts

## 3.1 Foundation

Composition contracts state who owns each resource, decision and fact when independent libraries meet in one application.

GOAL

### Make ownership and failures inspectable

A consumer can determine who configures, starts, cancels, persists, reconciles, and releases each resource. Failures preserve the facts needed for a meaningful next decision.

INVARIANT

### Composition preserves known effect evidence

A Composition boundary must not infer completed, prevented, or compensated work solely from cancellation intent, transport delivery, or loss of a reply. Unresolved implementation conflicts remain explicit gaps.

partial

PRINCIPLE

### Domain types and policy belong to callers

Applications supply native records and policy. Libraries require nominal types only when those types carry useful admission, identity, or ownership guarantees.

modeling

PRINCIPLE

### Bound work where it is owned

Limits state the work they bound and the party that enforces them. Bounded queues, finite waits, cancellation, and cleanup are separate guarantees.

robustness

The caller-value and extension rules are justified by ADR-0002 and ADR-0007 the bounding rules by ADR-0005.

## 3.2 Pending updates

ruling **A composed request budget needs explicit cleanup** 2026-10-06

Relay owns one admitted request deadline and a separate bounded cleanup interval; its package design owns implementation acceptance. Saga driver timeout is followed by bounded synchronous cleanup. A composed foreground deadline must reserve admission, execution, cancellation grace, and cleanup rather than assigning the same duration to every phase.

ruling **External schema breadth must be explicit** 2026-10-06

Blueprint admits a finite runtime schema profile. General third-party MCP discovery requires a chosen supported vocabulary, bounded validation, and safe provider projection; sibling-to-sibling discovery alone does not establish that breadth.

ruling **Durable scheduling integration remains application-owned** 2026-10-06

Applications drive and cancel durable Saga and Fabric work through Grind jobs. The intended broader scheduling and retention integration requires a decision about authority before any common driver abstraction is introduced. See ADR-0004.

verify **Carried-over caller contracts are unchecked against code** 2026-10-17

The caller-contract, execution-boundary, protocol and observation sections state rules moved from retired Oversight guides. Each rule needs a comparison against the owning package source before the section counts as verified.

## 3.3 Caller contracts

The public API conventions protect caller choices and extension guarantees. They constrain a boundary without prescribing an identical implementation for every package.

### 3.3.1 Values and definitions

#### Caller-owned values

**responsibility**

Preserve native inputs, outputs and meaningful failures through public boundaries.

**interface**

Typed definitions bind names, codecs and behavior once; caller-built configuration values use opaque constructors and composable settings.

**interactions**

A heterogeneous registry erases types only after binding each typed operation to its own definition. Persistence requests additional versioned codecs where durable replay needs them.

**invariants**

A required argument changes observable behavior. Already-owned names and codecs are derived rather than requested again. Missing, null, false and unknown remain distinct when their meanings differ.

**failure**

Rejected runtime input has a typed failure. Source definition mistakes follow the package's explicit programmer-error/check contract.

- Configuration construction does not open connections or start work. Complete configuration is validated before owned resources or external effects begin.
- Returned records are read by labelled fields. Growing outcome types retain meaningful classifications without erasing evidence into diagnostic strings.
- A static record codec, a runtime schema contract, a persistence codec, and a protocol declaration may share data but establish different guarantees. Small explicit adapters remain appropriate where those guarantees differ.

ADR-0002 ADR-0007

### 3.3.2 Extensions retain shared guarantees

#### Public extension boundary

contract

**Vary behavior without bypassing admission or ownership.**

**responsibility**

The library retains its common admission, bounds, correlation and cleanup rules; the extension supplies only its declared variable behavior.

**interface**

Public typed callbacks, protocol reducers, codecs or storage ports.

**interactions**

The consumer owns captured resources and the library owns the invocation lifetime it explicitly promises. Callback process affinity is part of that agreement.

**invariants**

An extension does not gain an undocumented path around destination policy, version checks, principal binding or state transitions.

**failure**

Unsupported extensions fail explicitly; a callback failure cannot be interpreted as confirmed non-execution unless that is known.

BEHAVIOR

### A new extension must work outside its library

**Given** A consumer supplies its own domain values and extension behavior.

**When** The consumer exercises the common and advanced public paths.

**Then** The extension reaches its capability without importing implementation internals.

**Then** Its failure and resource ownership remain observable through the public contract.

level: boundary area: Public extension admission

## 3.4 Execution and evidence boundaries

Several libraries execute work, but their authority applies to different facts. Composition retains those distinctions when mapping results or stopping work.

### 3.4.1 Delivery is not effect settlement

| Authority | Identity | Durable fact | What it cannot establish alone |
|---|---|---|---|
| Grind | Submission receipt and job/attempt identity | Admission and acknowledged delivery outcome | Whether an unacknowledged external effect occurred |
| Saga | Workflow run, step and effect identity | Checkpoint and known/unknown compensation evidence | Destination deduplication merely from supplying a key |
| Fabric | Run, operation, approval and child identity | Agent/graph state and recorded operation evidence | The real-world result of a lost tool reply |
| HTTP Gun / Relay / LLM Wire | Exchange or caller-propagated identity | Observed exchange and delivery evidence | Application effect completion or successful compensation |

### Outcome translation

contract

**Preserve the facts needed to recover across libraries.**

**responsibility**

The application maps delivery, domain failure, compensation and Effect evidence without treating them as synonyms.

**interface**

Typed package outcomes and explicit application policy.

**interactions**

Saga reports may be projected into Fabric tool results; LLM/HTTP failures may become Grind retry, snooze, discard or uncertain responses.

**invariants**

A later success does not erase an earlier unresolved effect. A cancellation request does not certify non-execution.

**failure**

Insufficient evidence leads to reconciliation or a truthful unknown result, not an automatic retry of a mutating effect.

BEHAVIOR

### Unknown delivery requires an effect policy

**Given** An application operation may have performed an external effect and lacks confirmation.

**When** The application maps the package failure into another execution authority.

**Then** The result retains the unresolved effect unless independent evidence settles it.

**Then** Retry requires an explicit idempotency, fencing, transaction or reconciliation policy.

level: boundary area: Outcome translation

ADR-0004

### 3.4.2 Identity scopes remain separate

- An **Admission identity** converges repeated admissions within its owning authority and retention window. It does not replace a destination's effect key.
- Grind uniqueness controls occupancy among selected job values and states. It does not serialize every external effect or prove permanent uniqueness.
- Fabric keyed invocation binds caller principal and key to a run; correlation remains an observation value, not the authenticated principal.
- Persistence codecs and worker/operation versions identify compatibility. An application changing behavior without its required version change cannot rely on replay equivalence.

### 3.4.3 Budgets and cleanup have different intervals

| Phase | Owner | Examples | Required composition fact |
|---|---|---|---|
| Admission | Endpoint/configuration authority | Token verification, context creation, queue/pool acquisition | Whether time and cancellation are bounded before effects start |
| Execution | Active exchange/attempt/run owner | HTTP body, model call, Saga attempt, Grind handler | Remaining budget and propagation into nested calls |
| Settlement | Effect authority | Undo, interrupted-effect resolution, tool settlement | Whether a returned stop still carries unresolved effects |
| Cleanup | Resource owner | Runner drain, claim release, stream close, pool shutdown | Whether return confirms release or only requests it |
| Durable waiting | Persisted run authority | Approval, graph signal, redelivery | Whether a caller timeout stops waiting, stops a driver, or cancels the durable run |

- The application reserves time for nested execution and cleanup rather than assigning the same nominal timeout at every layer. A per-attempt timeout does not bound retry delays or the whole conversation.
- Killing a BEAM worker ends local execution, not an external side effect. A borrowed resource remains available until its users have completed the cleanup guarantees they actually provide.
- A provider Retry-After longer than the foreground budget may be returned or scheduled durably. Clipping the delay and retrying early changes provider policy.

ADR-0005

### 3.4.4 Pool and process lifetimes are explicit

**Shared PostgreSQL lifetime** — contract

**Share one application database without merging recovery authorities.**

**responsibility**

Grind owns the pool it creates from caller configuration; application queries and the three PostgreSQL adapters borrow connections.

**interface**

Public connection accessors and borrowed transaction APIs; each adapter's migration and storage constructor.

**interactions**

Startup establishes schema compatibility before admitting work. Shutdown stops producers and dependent runtimes before releasing the pool.

**invariants**

A borrowed transaction is not committed by an enqueue helper. Claim fencing and destination effect fencing are distinct. Renewal connection reservations count toward deployment capacity.

**failure**

Lost commit acknowledgement requires receipt/reconciliation handling. Stopping the pool invalidates borrowers; a stopped caller may still require durable recovery.

## 3.5 Protocol, schema and authorization contracts

Protocol libraries validate wire meaning; applications decide domain meaning and trust. Schema and authentication reuse do not erase those responsibilities.

### JSON contracts through an HTTP boundary

contract

**Decode untrusted bytes into native application values.**

**responsibility**

HTTP Gun bounds and classifies the exchange; Blueprint validates admitted JSON and the application codec.

**interface**

A caller-owned HTTP client and record codec, with explicit response/parser bounds.

**interactions**

HTTP status and incomplete-body failure are inspected before choosing whether a complete document is required.

**invariants**

Schema validity, schema-profile support, codec decoding and business validation are separately identified. Exact-number refusal does not silently round a value.

**failure**

An oversized response can retain a successful status while lacking a usable document; the application determines whether that suffices for its task.

### MCP tools through an agent

contract

**Use protocol-neutral execution with explicit discovery policy.**

**responsibility**

Relay owns declaration and exchange semantics; Blueprint owns schema admission; Fabric owns tool policy and durable execution.

**interface**

Typed definitions or admitted dynamic declarations plus application-owned call/serve recipes.

**interactions**

Recipes propagate correlation and idempotency metadata and map complete tool-result evidence into the execution outcome.

**invariants**

Unsupported schemas fail visibly. A peer's read-only hint is not independent proof that a mutating effect is impossible.

**failure**

Unknown delivery remains uncertain unless an application-approved property of the tool justifies a safer classification. Buffered hosting exposes only the lifecycle information its host supplies.

### Authentication around a protocol boundary

contract

**Separate token validity from application permission.**

**responsibility**

Warden validates issuer/token properties and owns custody; Relay enforces MCP resource admission; the application maps claims into a principal and business permissions.

**interface**

Warden resource validation or introspection, Relay verifier/context ports, and Fabric approver proof construction.

**interactions**

Local signature validation and introspection have different freshness/network costs. A verifier's deadline and captured resources remain explicit.

**invariants**

A valid token does not grant every tool permission. Reviewer identity is derived from authenticated claims; only the configured verifier can authorize its approval proof.

**failure**

Authentication refusal, insufficient permission and unavailable validation infrastructure remain distinguishable; no failure opens the protected boundary.

BEHAVIOR

### Protocol verification does not grant domain permission

**Given** A caller presents a valid identity without the permission required by an operation.

**When** The caller requests the protected operation.

**Then** The application refuses the operation without beginning its business effect.

**Then** The refusal preserves the distinction between invalid identity and insufficient permission.

level: boundary area: Authorization composition

ADR-0008

### Email verification and application identity

contract

**Compose delivery and proof of possession without importing account policy.**

**responsibility**

Correio owns message admission, delivery evidence and challenge consumption. The application owns trusted account lookup, current permission and session publication.

**interface**

A typed sender, an explicitly configured challenge service, opaque verification evidence and an application-native principal mapping.

**interactions**

A capture sender and optional mailbox handler share one caller-owned capture. Email verification and Warden OIDC identity remain distinct authentication methods in the application's principal.

**invariants**

Accepted mail does not establish authentication. Inspecting a message does not consume its challenge. Only intentional confirmation and current account policy can produce an application session.

**failure**

Unknown submission is not automatically resent. Confirmed verification survives a session-publication failure as application-owned evidence; retrying publication does not verify a consumed answer again.

ADR-0017

## 3.6 Observation and diagnostic contracts

Observation joins work without controlling it. Applications decide which events to retain, export, and disclose.

### Cross-package observation

contract

**Associate related work without importing a tracing runtime.**

**responsibility**

Sinal owns the correlation value and typed observation boundary; applications select and propagate it.

**interface**

Explicit optional correlation on starts, client views and event metadata.

**interactions**

Agent, tool, Saga, job and HTTP observations carry the same application correlation where they belong to the same operation.

**invariants**

Correlation neither authorizes effects nor replaces durable identities. Native synchronous handlers can delay emitting work; bounded forwarding is an explicit ownership choice.

**failure**

Observation loss or handler failure is diagnostic evidence, not proof that durable work failed or committed.

- The application installs lifetime-scoped observers and routes. A process-local guess about attribution does not replace explicit propagation across asynchronous work.
- Diagnostic rendering excludes credentials and private application payloads unless the caller deliberately supplies a safe public projection. Typed evidence is retained before rendering.
- An exporter may map Observation correlation into a trace system. Exporter policy and parent/child span semantics remain outside the core correlation contract.
- Retrace reconstructs selected application state from safe checkpoints and messages. It does not replace Sinal correlation or infer business completion. Applications own deterministic reducers, disclosure policy and build-compatible renderers; the frontend and server introduce no mandatory ecosystem runtime.

ADR-0006

CONTEXT

# Evidence and validation

# 4 Evidence and validation

## 4.1 Foundation

Evidence states what an executed check establishes. Retained experiments answer named questions, and repository checks run from locked dependencies.

GOAL

### Validate observable behavior

Unit and integration tests execute public contracts with a pinned toolchain and disposable services. Formatting and compilation use standard tools; design validation uses the pinned shared checker.

GOAL

### Keep experiments tied to named questions

A Retained experiment states the question it answers, the cases it executes and the limits of its result.

NO-GOAL

### Historical reports as present contracts

Assessment chronology and decision rationale belong to ADRs. Test transcripts, release plans and obsolete signatures do not define the current design.

INVARIANT

### Evidence claims name their actual boundary

Compilation, scripted exchanges, native runtime tests, database tests and upstream conformance establish different claims. An unexecuted scenario is not reported as passing.

convention

PRINCIPLE

### Use executable consumers to judge interfaces

An ordinary task, an advanced case, native values and recovery must be expressible from a Public consumer. Internal elegance does not replace that evidence.

composition

The evidence-scoping decision is recorded in ADR-0009 and the source-resolution decision in ADR-0015.

## 4.2 Pending updates

ruling **A pending evidence marker cannot show an in-flight effect** 2026-10-07

A worker that stops after committing its pending marker and before sending leaves a condition that provider evidence alone can distinguish from an in-flight effect. The protocol reports uncertainty rather than inferring non-execution. See ADR-0020.

ruling **Application callbacks lack a remaining-deadline port** 2026-10-08

The admission experiment introduces no public remaining-deadline or cancellation argument for application callbacks. Such a port needs its own concrete consumer before a signature is chosen. See ADR-0022.

## 4.3 Validation commands

Nix runs shared validators directly. Small shell launchers run applications and experiments with their service fixtures.

CODE SH

```sh
nix flake check
nix run .#design-gate-check -- docs/design
nix develop --command bash scripts/test-applications.sh
nix develop --command bash scripts/test-experiments.sh
```

| Obligation | Check | Command group | Evidence and limit |
|---|---|---|---|
| Formatted authored files | Flake formatting check | Quality | Standard formatter diagnostics |
| Warning-free executable consumers | Gleam warnings-as-errors and separate Erlang `-Werror` compilation | Each application | Compiler diagnostics; no claim about dependency warnings |
| Local protocol, recovery and storage behavior | Each application's test suite, through the disposable database wrapper where required | Each application | Test and database logs; named scenarios only |
| Bounded compiler and oracle evidence | Retained experiment runners and protocol-pilot tests | Experiments | Positive controls, intended negative diagnostics and scoped oracle comparisons |

- The required commands run shared checks, applications and retained behavioral experiments. The final continuous-integration status requires every mandatory job to succeed; failures, cancellations and unexpected skips cannot satisfy it.
- The local application launcher discovers application manifests; continuous integration lists applications in its matrix. A new application must be added to that matrix and declare whether it needs PostgreSQL; no inventory checker enforces this convention.
- Handwritten Erlang in application source and tests compiles strictly after the Gleam dependency build.
- The experiments launcher runs every retained experiment except the Fabric reader check, which needs the older Fabric commit and the sibling commits recorded in that commit's own revision file. Continuous integration holds no sibling checkouts, so that check runs manually from a checkout with Git history. The applications and experiments are the usage examples, so a public API change that breaks one fails its test command.
- The interface laboratory and the HTTP declaration probe contain unfinished runtime declarations by design. Their positive and negative compilation tests are their acceptance boundary.
- Applications have no stable library-performance benchmark. Test durations are operational diagnostics, not throughput or latency claims; package benchmarks belong to the package that owns the workload.
- Required checks are standard formatting, compilation, behavioral unit and integration tests, and the imported shared design validator. README code excerpts use the shared pinned Snips package for source synchronization. Local documentation equality checkers, architecture conventions, source inventories, runner self-tests and evidence receipts are not validation obligations. Shared checker improvements belong upstream.

ADR-0028

## 4.4 Source qualification

A consumer's runtime code comes from its locked Git sources. Local-source experiments use explicit snapshots.

- Composition consumers declare Git sources with matching lockfiles. The Locked source is the commit a lockfile resolves for such a source, and historical readers keep their explicit revision selection. Editing a sibling working tree does not change consumer runtime code.
- The runner streams command output and propagates failures. Database logs survive cluster cleanup when a log directory is supplied. Missing services and failed tests remain failures; a custom provenance receipt is not required.
- All tracked library repositories are public, so dependency resolution needs no credential, and continuous integration persists none and passes none to test commands. Fresh dependency resolution verifies that the locked sources are available.
- Updating a library is a deliberate act: moving a lockfile reruns the affected consumers, and no scheduled job moves one. Inconsistent source requirements between libraries fail dependency resolution.
- A Local-source capture copies tracked and new authored sources into a fresh directory, refuses an existing directory, excludes credentials, local settings, caches and symlinks, and normalizes sibling dependencies throughout the copied graph. External sources and exact Hex oracle versions stay unchanged, and historical readers keep their explicit revision selection. The capture records revisions, dirty flags, original file hashes and overridden manifests. Its results do not qualify the locked sources.

- The Nix lock supplies the Gleam, Erlang/OTP, Elixir and PostgreSQL versions; only its lock input changes during a toolchain refresh. Rebar3 and Elixir use the shell's OTP package. After the first dependency build, repeated builds of an unchanged local-source consumer reuse its dependency resolution.
- Qualifying a coordinated library change updates consumer declarations and lockfiles together, then reruns the affected consumers. Oversight publishes no library artifact; release qualification combines these results with each library's own validation, and no workflow publishes packages or changes repository settings.

BEHAVIOR

### Consumers execute their locked sources

**Given** A consumer builds from its recorded declarations while a sibling working tree holds uncommitted edits.

**When** The consumer executes.

**Then** Its dependency resolver selects the locked sources.

**Then** The uncommitted sibling edits do not change its runtime code.

level: boundary area: Source qualification

ADR-0015

## 4.5 Experiments and acceptance evidence

Retained experiments answer narrow questions and stay separate from production libraries. Evidence is reproducible only when its inputs, oracle and execution boundary are explicit.

### Retained experiment

**responsibility**

Explore typed public contracts and failure boundaries without presenting a prototype as a production implementation.

**interface**

A runner, positive and negative cases, scripted peers and selected oracle comparisons inside each experiment directory, with a short README that gives its run command.

**interactions**

Experiments inform package-owned designs and ADRs. An accepted capability requires an implementation and a separate-consumer check in the owning repository.

**invariants**

Compiler success proves expressibility, not crash recovery or operational bounds. A pinned protocol snapshot retains its license and provenance. An expectation matrix is fixed before the implementation it judges.

**failure**

An incomplete experiment leaves its capability unproven; a stub, an expected failure or a skipped check is not counted as a working runtime.

| Retained system part | Question it answers | Limits of the evidence |
|---|---|---|
| `playground/relay_admission` | Do admission callbacks share the request deadline, and what changes when their process owner changes? | Barrier-driven public consumers check callback deadlines, disconnects, retiring capacity, shared budgets, shutdown, panic isolation and borrowed resources. Native process probes do not establish arbitrary resource cleanup or streaming lifecycle conformance. |
| `playground/interface_lab` | Can proposed native types, ownership states and protocol boundaries be expressed and distinguished? | Prototype behavior does not establish production lifecycle conformance. |

| Retained system part | Question it answers | Limits of the evidence |
|---|---|---|
| `playground/blueprint_compatibility` | How do codec and schema cases compare with the pinned implementation and validator? | Only executed supported-vocabulary cases establish equivalence. |
| `playground/ecosystem_pilot` | Can protocol declarations, model calls and typed observations compose through public imports? | A pilot does not replace each package's fault and resource tests. |
| `playground/archive_release` | Can authenticated admission and durable approval bind a release to its requester, revision and current permission? | Direct authorization admission and orderly store reopen do not establish full MCP transport or mid-effect crash recovery. |
| `playground/instrument_replay` | Can bounded recorded transport support offline evaluation under different application contracts? | Synthetic measurements and local faults do not qualify a scientific instrument or remote service. |
| `playground/erasure_campaign` | Can reversible preparation compose with irreversible delivery and forward reconciliation? | Application evidence, synthetic targets and local restarts do not establish legal erasure or host-loss guarantees. |
| `playground/fabric_record_compatibility` | Does a separately pinned earlier Fabric reader refuse the history record envelope? | Two public consumers preserve exact writer inputs and reader revision; refusal by one older reader does not establish every older release or backend fault case. |
| `apps/scripts/with-postgres.sh` | Can a consumer exercise real persistence in an isolated reproducible cluster? | No production database, remote provider or existing deployment is validated. |

- HTTP Gun owns the compiled public consumers for ordinary requests, streaming, offline playback and recording. Its separate-package compiler probes reject forged clients and string-bodied requests. The declaration-only HTTP API sketch is retired; its historical source remains in Git. ADR-0029

### 4.5.1 Interface laboratory

- The laboratory tests candidate typed APIs in one experimental package. Its modules are sketches and not production packages. Positive and negative consumers test compilation; independently distributable packages need consumer builds against actual package boundaries.
- A definition contains code and typed codecs. A snapshot contains only data that identifies the definition and records execution progress, never a closure, actor, socket or live lease. Restore receives a fresh definition and validates persisted data against it.
- A checkpoint capability distinguishes an execution that may be frozen from one still running. It does not prove exclusive ownership, because values can be copied; runner exclusion, commit ordering, stale revisions and signal deduplication need runtime checks.
- Approval is a suspension point and not a blocked worker. An uncertain external effect stays uncertain after freeze and thaw, because restoring a snapshot cannot establish whether a remote effect happened.
- Compiler checks reject incompatible types, distinguish execution capabilities and native child result types, and keep a pending identity out of a verified-identity interface. They cannot prove codec laws, graph validity, persistence safety, authorization, idempotency or scheduling.
- The experiment excludes full graph authoring, executable freeze and thaw, provider-specific continuation, streaming cancellation, full schema support, protocol conformance, upstream oracle suites and benchmarks.

### 4.5.2 Admission deadline and capacity

BEHAVIOR

#### One deadline covers admission and execution

**Given** An endpoint admits a request through a context callback and a token verifier.

**When** Admission consumes part of the request deadline.

**Then** The handler receives only the remaining budget and never a fresh one.

**Then** A positive decision that arrives after expiry cannot start the handler.

level: boundary area: Relay admission

BEHAVIOR

#### Retiring work retains capacity until it ends

**Given** A blocked admission callback outlives its request deadline.

**When** The endpoint answers the caller with a timeout and begins cleanup.

**Then** The callback's capacity slot remains occupied until the callback terminates, and other callers are refused for capacity meanwhile.

**Then** A late callback result never invokes the tool, and capacity recovers after termination.

level: boundary area: Relay admission

- A callback-created mailbox belongs to the worker that created it. A borrowed long-lived client, pool or actor is created outside the callback and stays application-owned.
- A timeout or process termination does not prove that an external effect did not occur. Detached children and children that trap exits do not become Relay-owned resources.
- Client header acquisition shares one request budget with transport; an acquisition that is blocked past its budget reports a timeout that was not sent, and cancellation, owner close and caller death stop the callback.

ADR-0022

### 4.5.3 Authorization and approval

BEHAVIOR

#### Approval binds the original requester and revision

**Given** A release awaits approval and a different authenticated principal later holds permission on the same document.

**When** The release executes with fresh application context.

**Then** The release is refused because the requester changed, and nothing is written.

**Then** A document revision that changed during approval is refused for the same reason.

level: boundary area: Archive release

- A completed agent conversation can contain a denied action. Business success is read from the action outcome and the application's release record, never from conversation completion.
- Permission is read from fresh application state at execution. A valid token and a valid approval proof do not carry stale permission forward.

ADR-0021

### 4.5.4 Irreversible effects and replay

BEHAVIOR

**Evidence is recorded before an irreversible send**

**Given** A campaign must delete several targets irreversibly after reversible preparation.

**When** A delivery reaches a target whose evidence is pending, confirmed or rejected.

**Then** Only the delivery that recorded pending evidence first may send the deletion.

**Then** Confirmed evidence returns its receipt and rejected evidence declines, both without another deletion; pending evidence reports uncertainty.

level: boundary area: Erasure campaign

BEHAVIOR

**Transport failure stays distinct from contract failure**

**Given** A recorded acquisition is replayed offline with two schema policies.

**When** A response is truncated, oversized, or complete with a value of the wrong type.

**Then** Truncation and excess size remain transport failures with their status evidence, and the wrong type is a located contract failure.

**Then** Independent replay runs keep their own cursor and correlation over one unchanged capture.

level: boundary area: Instrument replay

- Saga serves reversible preparation, and final irreversible effects use forward reconciliation with application evidence rather than invented compensation.
- A lookup result must equal the exact expected receipt. An absence, a malformed receipt or a failed lookup never authorizes replay.
- Cancellation stops future work and cannot restore data already deleted.

ADR-0020 ADR-0023

BEHAVIOR

**A consumer proves only executed behavior**

**Given** A consumer compiles but its persistence or external-effect scenario is skipped.

**When** Its result is used to assess a package contract.

**Then** The evidence is limited to compilation and the scenarios actually executed.

**Then** The unexecuted operational guarantee remains unverified.

level: boundary area: Evidence acceptance

- A Behavioral oracle names its version, license, relevant behavior and comparison limits. Native telemetry, JSON Schema, Gun/Cowlib, Reactor, Oban and protocol authorities are scoped evidence rather than automatic parity claims. The oracle reference holds each pin, what is compared, how outcomes are classified, how to run the comparison and where the claim ends.
- Open-source Oban behavior and commercial Pro behavior require separate contracts and lawful evidence. Certification of an upstream identity library does not certify its wrapper.
- Deterministic barriers expose races and ownership; sleeps alone do not establish event ordering. Real database tests complement pure transition tests.
- The authored behavior areas are public facade, extension admission, effect translation, durable identity, budget and lifetime, authorization, observation and evidence acceptance. Each application adds its domain-specific cases in its executable tests.

ADR-0009 ADR-0025 ADR-0027

## 4.6 Design maintenance and rendering

Repository-local design sources are the present contract. The pinned renderer checks structure and references while semantic review checks the contract against the system.

### Documentation authority

**responsibility**

Keep package-local design, canonical terminology and decision rationale discoverable without duplicate planning authorities.

**interface**

Typst design and glossary files, Markdown ADRs and coverage map, one generated layer document, and forwarded Nix render, check and context apps.

**interactions**

The README and agent instructions point to the local layer. Oversight links package owners and records ecosystem-wide decisions; operational guides link the relevant design rather than restating it.

**invariants**

The design is timeless. ADR rationale is evidence-backed; missing historical context is explicit. The Rendered layer is generated and never edited directly.

**failure**

A stale rendering or broken semantic reference fails the design gate. A green gate does not prove implementation conformance or correct design.

- Coverage classifies actual repository parts as captured, standard, or out-of-scope. A deleted component does not survive as an invented coverage row.
- Source and behavior changes require reviewing affected contracts, examples, callers and stored data. API similarity alone is not a reason to impose an ecosystem-wide migration.
- Current gaps remain in the owning pending ledger. ADRs explain decisions and alternatives; they do not replace the standing design.
- Package and nested-adapter gates use the same pinned schema and renderer bundle. Nested adapter layers are checked through the parent's declared nested-project path.
- Examples and fixtures keep a short README that gives what the directory is and how to run it, and the compiled applications and experiments are the usage documentation. Oracle pins and comparison limits live together in the oracle reference; narrative, results and research chronology live in ADRs or the design.

ADR-0001 ADR-0026 ADR-0027

# Glossary

Every term the layer declares, with the context that owns it.

## Admission identity

owned by composition

A caller-supplied or retained identity that lets an authority recognize repeated requests for the same admission. It does not prove that an external effect is unique.

## Applied marker

owned by applications

An application-owned durable record that an outcome has been incorporated. It prevents a later delivery from starting the same work again after execution records are pruned.

## Behavioral oracle

owned by evidence

A pinned external implementation or specification used to judge a stated subset of observable behavior. Its authority does not extend beyond that subset.

## Borrowed resource

owned by composition

A pool, client, actor or transaction that one party owns and another party uses without controlling its lifetime.

## Business identity

owned by applications

The application-chosen identity of one business operation. It is independent of job, workflow and queue identities and outlives their retention.

## Capability scope

owned by composition

The set of responsibilities a package is intended to support, including explicitly unresolved or unbuilt capabilities.

## Composition application

owned by applications

An independently compiled application that exercises several libraries through their public contracts.

## Composition boundary

owned by composition

An application-owned translation between independently meaningful package contracts.

## Consumer recipe

owned by composition

Application-owned code that translates between public package contracts without importing package internals.

## Effect evidence

owned by composition

Knowledge about whether an application effect was prevented, completed, compensated, or remains unresolved. Delivery acknowledgement alone does not establish business completion.

## Local-source capture

owned by evidence

A disposable copy of authored sources in which sibling dependencies resolve to the copied trees, used to exercise unpublished coordinated changes. It does not qualify the locked sources.

## Locked source

owned by evidence

The commit that a consumer's lockfile resolves for an unpublished library. The lockfile is the only record of that revision.

## Observation correlation

owned by composition

An application-selected value carried across observations so events can be associated with one operation. It neither authorizes work nor identifies a durable effect.

## Package authority

owned by composition

The library that owns a contract's meaning, admitted state transitions, and failure guarantees.

## Public consumer

owned by evidence

A separate package that imports only the public modules of the library being assessed.

## Reconciliation decision

owned by applications

An attributed application record that settles a finished unresolved operation from authoritative evidence, identified by one stable decision identity.

## Rendered layer

owned by evidence

The single generated document assembled from a repository's design sources and canonical vocabulary.

## Retained experiment

owned by evidence

A disposable-system prototype kept as executable evidence for a named design question, rather than as a production implementation.

## Scripted peer

owned by applications

A deterministic loopback service that stands in for an external provider and exposes planned failure paths without credentials.

## Session publication

owned by applications

The application step that turns confirmed verification evidence into an application session, identified by a publication identity rather than a browser credential.

## Source provenance

owned by evidence

The record of the package declarations, lockfiles and toolchain a consumer executed, kept separate from the repository's own revision and dirty flag.

## Unresolved effect

owned by composition

An application effect that may have occurred and lacks confirmation of either outcome.

## Validation profile

owned by evidence

A named subset of the repository's checks (quality, application, experiments, full) that the runner executes as one unit.